PROJECT DOCUMENTATION

SMART TRAFFIC SIGNAL SCHEDULING SYSTEM

Using Circular Linked List

Project Type	Data Structures and Algorithms (DSA) Mini Project
Programming Language	C++
Core Data Structure	Circular Linked List
Application	Educational traffic-signal scheduling simulation

Prepared as academic documentation for a DSA mini project. Student details and college information may be added before submission.

CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope of the Project
6. Hardware and Software Requirements
7. Data Structure Overview
8. System Design and Working
9. Algorithm
10. Pseudocode
11. C++ Implementation Overview
12. Sample Input and Output
13. Time and Space Complexity
14. Advantages
15. Limitations and Future Enhancements
16. Test Cases
17. Conclusion
18. References

1. Abstract

The Smart Traffic Signal Scheduling System is a C++ DSA mini project that demonstrates how a circular linked list can manage a repeating sequence of traffic signals. Each node stores a road or signal name, a green-light duration, and a pointer to the next node. The last node links back to the first node, allowing the schedule to repeat without restarting traversal manually. The project accepts signal details from the user, displays the sequence, and simulates signal-by-signal scheduling. It is intended for learning linked-list operations and scheduling logic; it does not control real traffic infrastructure.

2. Introduction

Traffic intersections need an organized sequence to decide which road direction receives a green signal. A repeating sequence can be modeled with a circular linked list because the final signal naturally leads back to the first signal. This project applies the circular linked-list concept to a simple traffic-signal simulation. It demonstrates node creation, pointer connections, traversal, and repeated processing in a practical scenario.

3. Problem Statement

Design and implement a Smart Traffic Signal Scheduling System using a circular linked list. The program should accept the number of signals, their names, and their green-light durations; create one node per signal; connect the nodes in the intended order; and link the final node back to the head node. It should display the configured sequence and simulate activation of each signal in turn. After a signal's turn, the program should advance to the next node and repeat the cycle until the user requests termination.

4. Objectives

- Represent each road signal as a node in a circular linked list.
- Store the signal name and green-light duration in each node.
- Create and connect nodes in the user-defined order.
- Display the signal sequence and active signal.
- Move to the next signal after each simulated interval.
- Repeat from the final node to the first node.
- Provide a simple way to stop the scheduling simulation.
- Demonstrate circular traversal and dynamic memory use in C++.

5. Scope of the Project

The project focuses on software-level scheduling and linked-list behavior. It can be run in a C++ compiler or IDE and may use console output to show the current signal. Signal durations are configuration values. The basic implementation is suitable for academic demonstration and can later be extended with a real timer, menu-based management, or simulated traffic inputs.

6. Hardware and Software Requirements

Hardware

- Desktop or laptop computer.
- Keyboard and monitor.

Software

- Operating system: Windows, Linux, or macOS.

- C++ compiler supporting C++11 or later (for example, GCC, MinGW, or an IDE with a C++ compiler).
- Text editor or IDE such as VS Code, Code::Blocks, or Visual Studio.

7. Data Structure Overview

Circular Linked List: A linked list in which the next pointer of the last node points to the first node rather than NULL. Each node in this project contains a signal name, a green duration, and a next pointer.

Head pointer: Points to the first signal in the sequence. **Current pointer:** Identifies the signal currently being simulated. The list is traversed using the next pointer, and the last node returns traversal to the head.

Node representation:

```
[Signal Name | Green Duration | Next Pointer] → [Next Signal | Duration | Next Pointer] …
→ back to Head
```

8. System Design and Working

- Input: Read the number of signals and the name and duration for each signal.
- Node creation: Create a node for every signal.
- Linking: Attach each new node to the end of the list and keep the final node linked to the head.
- Display: Traverse the list once to print all signal names and durations.
- Scheduling: Start from the head, display the active signal and its duration, then advance to current->next.
- Repeat: Continue cycling through the nodes until the user enters the stop command.

Example cycle: North Road → East Road → South Road → West Road → North Road. The cycle repeats because the final node points to the first node.

9. Algorithm

1. Start.

2. Initialize head = NULL.

3. Read the number of traffic signals n; validate that n is positive.

4. For each signal, read its name and green duration; validate the input.

5. Create a node and append it to the circular linked list.

6. Ensure the last node's next pointer points to head.

7. Display the signal sequence by traversing until the pointer returns to head.

8. Set current = head.

9. Display current signal as GREEN and show its duration.

10. After the interval/simulation step, mark the signal inactive and set current = current->next.

11. Repeat scheduling until the user requests stop.

12. Free allocated nodes and end.

10. Pseudocode

```
START
  Read n
  IF n <= 0: print error; STOP
  head = NULL
  Repeat n times:
      Read signalName and greenTime
      Create newNode(signalName, greenTime)
      Insert newNode at end of circular list
      Ensure lastNode.next = head
  Display all signals
  current = head
  WHILE user has not requested stop:
      Display current.signalName as GREEN
      Display current.greenTime
      Wait for interval / user input
      Display current signal as inactive
      current = current.next
  Free all nodes
STOP
```

11. C++ Implementation Overview

The implementation uses a **Node** structure to store the signal name, duration, and next pointer. A **TrafficSignal** class manages the head pointer and provides functions to add a signal, display the sequence, run the scheduling simulation, and release allocated memory. The scheduling loop advances with current = current->next. In the interactive version, pressing Enter advances to the next signal and entering q stops the simulation; the duration is displayed but is not automatically timed.

The code provided with this documentation should be saved separately as a .cpp source file. This report explains the design and expected behavior rather than embedding the full source code.

12. Sample Input and Output

Sample input

```
Number of signals: 3
Signal 1: North Road, duration: 30 seconds
Signal 2: East Road, duration: 25 seconds
Signal 3: South Road, duration: 20 seconds
```

Sample output

```
SMART TRAFFIC SIGNAL SCHEDULING SYSTEM
Using Circular Linked List

--- Traffic Signal Sequence ---
North Road | Green time: 30 seconds
East Road | Green time: 25 seconds
South Road | Green time: 20 seconds

Active Signal: North Road
Status: GREEN
Duration: 30 seconds

North Road signal is now RED.
```

The sequence then advances to East Road, South Road, and back to North Road. Exact console prompts may vary depending on the final source-code version.

13. Time and Space Complexity

Let n be the number of traffic signals.

- Insertion at the end as implemented by traversing from head each time: O(n) per insertion; creating all n nodes this way takes $O(n^2)$ total time.
- Displaying all signals: O(n), because each node is visited once.
- Advancing to the next signal: O(1), because the next pointer is followed directly.
- Space complexity: O(n), because one node is stored for each signal.

14. Advantages

- Naturally supports repeating signal sequences.
- The final signal links directly back to the first signal.
- The number and names of signals can be configured by the user.
- Demonstrates a practical use of dynamic memory and pointer traversal.
- Moving to the next signal takes constant time.

15. Limitations and Future Enhancements

Limitations

- This is a simulation, not a real traffic-light controller.
- The basic version uses a fixed sequence and predefined durations.
- It does not measure live traffic density or prioritize emergency vehicles.
- In the interactive implementation, the user advances the sequence manually; the duration is displayed rather than automatically timed.

Future enhancements

- Add an automatic timer to switch signals after the configured duration.
- Allow users to add, delete, or update signals through a menu.
- Simulate traffic density and adjust durations using defined rules.
- Add pedestrian-crossing phases and emergency-vehicle priority in a safe simulation.
- Create a graphical interface showing signal colors and countdown timers.

16. Test Cases

Test	Input / Condition	Expected Result
1	Three valid signals	All three signals are stored and displayed in order.
2	Last node traversal	After the last signal, traversal returns to the first signal.
3	Number of signals = 0	Program displays an invalid-input message and exits.
4	Duration = 0 or negative	Program rejects the invalid duration.
5	User enters q during scheduling	Scheduling stops and allocated nodes are released.

17. Conclusion

The Smart Traffic Signal Scheduling System demonstrates how a circular linked list can represent a repeating sequence of traffic signals. By storing each signal in a node and linking the last node back to the first, the program can cycle through the signals continuously and efficiently. This project reinforces core DSA concepts such as node creation, pointer management, traversal, dynamic memory allocation, and circular linked-list applications.

18. References

- C++ language reference and standard-library documentation: https://en.cppreference.com/
- Data Structures and Algorithms course notes on linked lists and circular linked lists.
- The project's problem statement, algorithm, and C++ implementation prepared for this academic mini project.